

Design and Analysis of Algorithms for Identifying Frequent and Less Frequent Items

Marko Pongrac

Departamento de Eletrónica, Telecomunicações e Informática

mpong rac@ua.pt

Abstract – This report investigates methods for counting items in data streams, focusing on both exact and approximate approaches, and on identifying the most frequent items in a dataset. Three algorithms are implemented and analyzed: an exact counter, a probabilistic counter with a fixed probability of 0.125, and the Space-Saving Count algorithm for frequent item detection. The quality of the approximate estimates is evaluated by comparing them with exact counts using relative deviations from the mean count and the frequency of occurrence of all items. In addition, each method’s ability to correctly identify the most frequent items and maintain their relative ordering is assessed. Along with accuracy, computational efficiency and memory usage are also evaluated. The results demonstrate the trade-offs between accuracy, memory consumption, and scalability, highlighting the practical applicability of approximate counting algorithms for large data streams. Although both approximate methods are viable alternatives to exact counting, their memory efficiency could be further enhanced in more optimized implementations or lower-level programming frameworks.

I. INTRODUCTION

The problem of identifying frequent and less frequent items is a fundamental problem in data analysis, with wide range of applications, especially in recommendation systems and large-scale data mining. Since the amount of data is increasing exponentially each year, storing and processing exact frequency information for all items in large datasets may be computationally intractable or even infeasible due to memory and time constraints. This has motivated the development of approximate counting techniques that trade exactness for memory efficiency.

While exact counting techniques guarantee correct frequency values for each distinct item (class) that we count, their memory requirements grow linearly with the number of unique elements, limiting their scalability in high-volume datasets and data streams.

This report explores and compares three different approaches for item frequency estimation: the exact counter, the probability counter and Space-Saving Count to identify the most frequent items. The probabilistic counters increment the item count with fixed

probability set to 0.125, significantly reducing the memory consumption while introducing the estimation variance. Additionally, the Space-Saving algorithm [1] is employed to identify the most frequent items in a stream using a bounded number of counters, offering strong guarantees for the detection of the k -most frequent items in the dataset.

The designed algorithms are evaluated on a real-world dataset, downloaded from Kaggle, and their performance is analyzed in terms of accuracy, memory usage, and computational efficiency. Exact count is used as a reference to determine the quality of the approximate counting methods using absolute and relative error metrics. Furthermore, the stability of the results obtained by approximate methods is evaluated across multiple independent runs. The experimental analysis highlights the trade-offs between accuracy and resource usage when processing large-scale data streams.

II. ALGORITHMS

A. Exact Counter

The **Exact Counter** algorithm provides a simple way to compute the frequency of each unique item in a dataset. For every item in the input stream, the algorithm maintains a dictionary that maps items to their occurrence counts. Whenever an item is encountered, its corresponding counter is incremented. This method guarantees accurate results but requires memory proportional to the number of distinct items. Additionally, the memory footprint can be measured to assess the computational resources required for storing all counters. It is important to note that all of

Algorithm 1 Exact Counter for Item Frequencies

```

1: Input: Stream of items  $S = [s_1, s_2, \dots, s_n]$ 
2: Output: Frequency dictionary  $C$  mapping items to counts
3: Initialize empty dictionary  $C \leftarrow \{\}$ 
4: for all  $s \in S$  do
5:   if  $s \in C$  then
6:      $C[s] \leftarrow C[s] + 1$ 
7:   else
8:      $C[s] \leftarrow 1$ 
9:   end if
10: end for
11: return  $C$ 

```

the counters were stored by using an ordinary **int** data type in python, therefore all of them should have the same memory size, in case that the counters do not reach extremely high values.

B. Probabilistic Counter

The Probabilistic Counter algorithm provides an efficient way to estimate item frequencies in a data stream while reducing memory usage. Instead of incrementing the count for every occurrence, each item is incremented with a fixed probability P . All counters start as 8-bit integers (uint8) to minimize memory consumption. If a counter reaches its maximum value, it is dynamically cast to a larger integer type (uint16 or uint32) to avoid overflow. This approach allows the algorithm to handle both frequent and infrequent items efficiently while keeping the memory footprint smaller than exact counting in many practical scenarios.

Algorithm 2 Probabilistic Counter with Dynamic Integer Sizes

```

1: Input: Stream of items  $S = [s_1, s_2, \dots, s_n]$ , probability  $P$ 
2: Output: Approximate counter  $C$  mapping items to counts
3: Initialize empty dictionary  $C$  with default type uint8
4: for all  $s \in S$  do
5:   if  $\text{random}() \leq P$  then
6:      $val \leftarrow C[s]$ 
7:     if  $val$  is max for uint8 then
8:        $val \leftarrow$  upgrade to uint16
9:     else if  $val$  is max for uint16 then
10:       $val \leftarrow$  upgrade to uint32
11:     end if
12:      $C[s] \leftarrow val + 1$ 
13:   end if
14: end for
15: return  $C$ 

```

By using this approach of "dynamic memory increase", items with lower frequency should have much smaller memory usage, compared to the items that have much higher frequencies. Therefore, the total memory usage should in theory be much lower compared to the ordinary counter.

C. Space-Saving Count

The **Space-Saving Count** algorithm provides an efficient method to estimate the most frequent items in a data stream while severely limiting the memory usage. Instead of storing counts for all unique items, the algorithm maintains up to k counters, each associated with an item, and uses a min-heap to track the item with the smallest count. For each processed item, if it is already being tracked, its counter is simply incremented by 1. If the item is not yet tracked and there is an available counter, a new counter is initialized with an initial count of 1. If all counters are currently being

used, the algorithm replaces the item with the smallest count with the new item, setting its count to the previous minimum and incrementing its value by 1.

This strategy allows Space-Saving to efficiently identify most frequent items in large streams without storing all elements, significantly reducing memory usage compared to exact counting. Although the counts are approximate, the algorithm ensures that the estimated count of each item is close to its true frequency, particularly when the number of counters is sufficiently large relative to the number of unique items in dataset.

Algorithm 3 Space-Saving Algorithm for Frequent Items

```

1: Input: Stream of items  $S = [s_1, s_2, \dots, s_n]$ , number of counters  $k$ 
2: Output: Dictionary  $C$  mapping items to estimated counts
3: Initialize empty dictionary  $C \leftarrow \{\}$ 
4: Initialize empty min-heap  $H \leftarrow []$  to track item with smallest count
5: for all  $i \in S$  do
6:   if  $i \in C$  then
7:      $C[i] \leftarrow C[i] + 1$ 
8:      $\text{heapify}(H)$ 
9:   else if  $|C| < k$  then
10:     $C[i] \leftarrow 1$ 
11:     $\text{push}(1, i)$  to  $H$ 
12:   else
13:      $(\text{min\_count}, \text{min\_item}) \leftarrow$  pop minimum from  $H$ 
14:      $\text{delete } C[\text{min\_item}]$ 
15:      $C[i] \leftarrow \text{min\_count} + 1$ 
16:      $\text{push}(\text{min\_count} + 1, i)$  to  $H$ 
17:   end if
18: end for
19: return  $C$ 

```

Here, it is important to note that, unlike the **Probabilistic Counter**, the implementation of the **Space-Saving Count** algorithm did not dynamically increase the memory allocation of the counters when they reached their maximum value. Instead, all k counters were implemented using Python's standard **int** data type, which can grow arbitrarily large, avoiding overflow issues without explicit type upgrades.

III. FORMAL COMPLEXITY ANALYSIS

A. Time Complexity

The time complexity of the **Exact Counter** algorithm is linear with respect to the number of processed elements. The main loop is executed n times, where n denotes the total number of items in the data stream. For each item, a constant-time $O(1)$ dictionary update is performed, since Python **dict** objects are optimized for such operations. Therefore, the overall time complexity of the algorithm is $O(n)$.

The **Probabilistic Counter** algorithm also processes each stream item once and performs a counter update

TABLE I
TIME AND SPACE COMPLEXITY OF FREQUENCY COUNTING ALGORITHMS

Algorithm	Average-case Time Complexity	Space Complexity
Exact Counting	$O(n)$	$O(m)$, where m is the number of unique items
Probabilistic Counter	$O(n)$	$O(m)$, where m is the number of unique items
Space-Saving	$O(n \cdot k)$	$O(k)$

with a fixed probability $P = 0.125$. Since dictionary access and type checking are constant-time operations, the expected time complexity remains $O(n)$. The additional operations introduced by occasional type upgrades occur infrequently and do not affect the asymptotic behavior of the algorithm. Consequently, the overall time complexity is still dominated by the linear scan of the data stream, resulting in an expected time complexity of $O(n)$.

The **Space-Saving Count** algorithm processes each element of the data stream exactly once. Let n denote the total number of elements in the stream and k the maximum number of maintained counters.

For each item in the data stream, one of the following operations is performed. If the item is already present among the monitored counters, its count is simply incremented by 1, resulting in a constant $O(1)$ time complexity for a simple dictionary lookup. In this implementation, the heap structure is rebuilt using a **heapify** operation, which requires $O(k)$ time.

If the item is not currently monitored and fewer than k counters are currently used, a new counter is created and inserted into the heap. Here, insertion dominates the complexity polynomial with $O(\log k)$ time.

If all k counters are already in use and a new item is being processed, the algorithm removes the item with the smallest count and replaces it with the currently processed item. This involves a heap removal and insertion, both of which take $O(\log k)$ time, resulting in the same total time complexity.

As a result, the worst-case time complexity per processed item is $O(k)$ due to the heap reconstruction step. Since each of the n elements in the data stream is processed once, the overall time complexity of this implementation is:

$$O(n \cdot k)$$

However, since the whole purpose of the algorithm is to count using the small number of counters, the parameter k is typically small, making the algorithm time efficient for large data streams. It is possible to achieve $O(n \log k)$ time complexity with smart heap management, but such optimizations were not part of this research.

B. Space Complexity

The space complexity of each algorithm depends on how the counters are maintained and the data structures used. In **Exact Counting**, a separate counter storing *int* data type is maintained for each unique item in the stream. If the stream contains n unique items, the space complexity is $O(n)$, which can become

prohibitive for large streams with high cardinality.

In the case of the **Probabilistic Counter**, a counter is still maintained for each unique item, but each counter is stored in a smaller data type, such as *numpy.uint8*. Therefore, the space complexity remains $O(n)$, where n is the number of unique items, although the memory footprint per counter is significantly reduced, typically by a factor of 0.5 or 0.25 compared to using standard integers.

For the **Space-Saving Algorithm**, only k counters are maintained, along with a min-heap to track the item with the smallest count. The space complexity is $O(k)$, where k is typically much smaller than the number of unique items in the stream. This makes the algorithm particularly suitable for high-cardinality data streams while keeping memory usage low.

IV. EXPERIMENTAL METHODOLOGY

For the experiments, a dataset of real estate listings in Portugal was used, obtained from Kaggle¹. The dataset contains information about towns and properties, which was processed to extract the stream of town names for frequency analysis. Each algorithm—Exact Counting, Probabilistic Counter, and Space-Saving—was applied to the same data stream to ensure a fair comparison in terms of estimated counts and memory usage. The experiments were performed using 64-bit Python and executed on an 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz processor. To estimate the amount of used memory, the **pympler** [2] Python module was used.

The performance of Probability Counter algorithm was evaluated using several statistical metrics, including the mean, median, and standard deviation of the estimated counts across all towns. These metrics provide insights into the typical behavior of the algorithms (mean), the central tendency (median), and the variability of the estimates (standard deviation), allowing for a more comprehensive comparison of accuracy and stability. In order to measure the stability of the counter, 1000 independent runs of the algorithm were performed.

For the Space-Saving Count algorithm, performance evaluation was more focused on the analyzing the influence of the number of counters k on the top 5 most frequent items and their count. The main motivation behind that decision was to analyze how does the number of counters influence the accuracy of the algorithm. The values of k started from 5, since the analysis was

¹<https://www.kaggle.com/datasets/luvathoms/portugal-real-estate-2024>

centered on the top 5 items, and were increased in equal steps of 5 up to $k = 2200$. Therefore, the tested values of k ranged from 5 to 2200, allowing a detailed examination of the trade-off between memory usage and accuracy across a wide range of counter configurations.

V. RESULTS

A. Exact Counting

As a baseline, the Exact Counting algorithm was applied to the dataset to determine all of the important metrics for the towns: unique number of towns, frequency of each one, memory usage.

The total number of items in the dataset was 135534, with number of unique towns being 2263. The top five most frequent towns observed in the dataset are shown in the Table II. We can see that none of the towns has the frequency higher than 2228, the number that is in range of `uint16`, which can store the number from 0 to 65535. Therefore, we can conclude that at least half of the bytes assigned to our counters are never used, since standard `int` stores 4 bytes, double the amount of `uint16`. The total used memory is 258704 bytes, since the Python `int` data types also include overheads and stores additional data, unlike the same data types in C or C++.

TABLE II
TOP 5 MOST FREQUENT TOWNS BY EXACT COUNT

Town	Count
Paranhos	2228
Cascais e Estoril	1459
Albufeira e Olhos de Água	1433
C., S. Ildefonso, Sé, M., S. Nicolau e Vitória	1069
Portimão	1054

These results serve as the ground truth for comparison with the approximate counting algorithms (Probabilistic Counter and Space-Saving), allowing evaluation of both accuracy and memory efficiency.

B. Probability Counter

For the Probability Counter, it is important to note that, while comparing the results with baseline results obtained by Exact Counter, mean and median value should be multiplied by 8, since we were counting each item with the probability of 0.125. Table III shows the statistics of the 5 most frequent items in the dataset. As we can see, the scaled mean value does not diverge much from the values obtained by the exact counter, with none of them having the difference higher than 2. Translated to the relative error, that means that for frequent items the relative average error of the average compared to the exact value is less than 1%.

For these top 5 towns, the standard deviation (SD) values indicate the variability of the probabilistic counts across the 1000 runs. While the counts for frequent towns like Paranhos show relatively low SD compared to their scaled mean, less frequent towns have

slightly higher relative variability. This demonstrates that the Probabilistic Counter provides stable and consistent estimates for the most common items, while the variability naturally increases for less frequent ones.

However, when evaluating the consistency of the Probabilistic Counter, it is important to consider how many different towns are counted in each individual run, as extremely rare items might be completely skipped in some runs. Figure 1 illustrates the distribution of the occurrences of each unique town in all of the runs. As we can see, while all of the towns appeared at least 100 times, out of possible 1000, only 56.34% were recorded at least 900 times and only 23.91% were recorded in each run. This further proves that, while the Probabilistic Counter is reliable when counting frequent items, it still struggles with less frequent items.

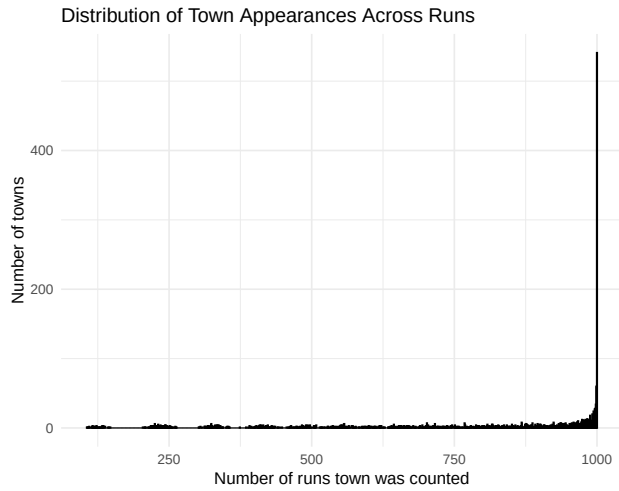


Fig. 1 - Distribution of town appearances across 1000 runs of the Probabilistic Counter. The plot shows in how many runs each town was counted. Frequent towns are recorded in nearly all runs, while rare towns may appear only in a few.

In addition to accuracy, it is important to evaluate the memory usage of the Probabilistic Counter. Across 1000 runs, the mean memory consumption was approximately 273,445 bytes (about 267 kB), with a median value of 273,448 bytes (267 kB), indicating a very stable memory footprint. The minimum observed usage was 268,080 bytes (261.8 kB), while the maximum reached 278,568 bytes (272 kB). These small variations reflect minor differences in internal data structures or dynamic allocation of counters. Overall, the Probabilistic Counter demonstrates highly predictable and efficient memory usage, but it did not show less memory usage compared to the Exact counter, probably because of the way the Python stores its primitive data types.

C. Space-Saving Count

The **Space-Saving Count** algorithm was evaluated by varying the number of counters k from 5 to 2200 and observing both memory consumption and accuracy with respect to top-5 most frequent towns determined by the baseline model.

TABLE III
TOP 5 TOWNS BY SCALED MEAN COUNT FOR PROBABILISTIC COUNTER

Town	Mean count	Median count	SD count	Scaled mean
Paranhos	278.0	278.0	15.2	2227.0
Cascais e Estoril	182.0	182.0	12.6	1460.0
Albufeira e Olhos de Água	179.0	180.0	12.5	1432.0
C., S. Ildefonso, Sé, M., S. Nicolau e Vitória	133.0	133.0	10.8	1067.0
Portimão	132.0	132.0	10.8	1053.0

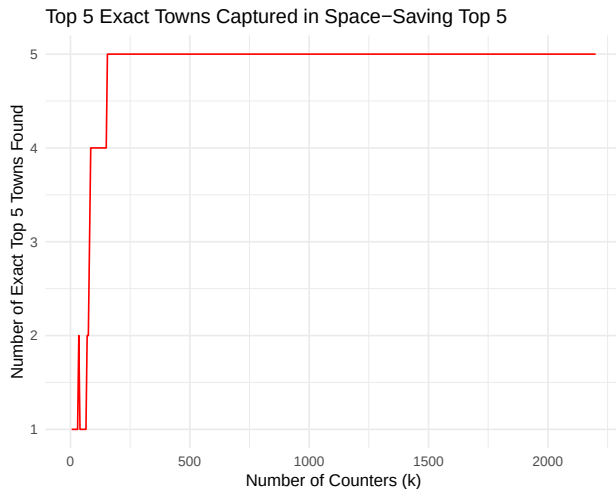


Fig. 2 - Number of exact top-5 towns captured by the Space-Saving algorithm for different values of k . The maximum value of 5 indicates that all of the towns were captured.

Figure 2 shows that for small values of k , the algorithm may fail to capture all of the most frequent items. However, as k increases, the number of correctly identified top-five towns rapidly improves, eventually reaching full agreement with the Exact Counter. Interestingly, the algorithm seems to correctly detect 2 most frequent items out of 5 for $k = 35$, but then again drops to only 1 hit, until k reaches the value of 70. However, the accuracy of the algorithm then significantly increases, starting to correctly assess top-5 most frequent for all the values of k equal or greater than 155.

It is also important to note that the total number of unique items in the dataset was 2263. This implies that high precision for identifying the top-five most frequent items can be achieved by using a number of counters corresponding to only approximately 6.8% of the total number of unique items, highlighting the efficiency of the Space-Saving algorithm.

Figure 3 illustrates the relationship between the number of maintained counters and memory usage. As expected, memory consumption increases approximately linearly with k , since the algorithm stores one counter per tracked item. However, we can also notice a significant memory drop once the number of counter becomes 2000, before the memory usage once again exhibits linear growth. The dashed horizontal line denotes the memory usage of the Exact Counter, highlighting the point at which the Space-Saving approach becomes more memory-efficient than exact counting.

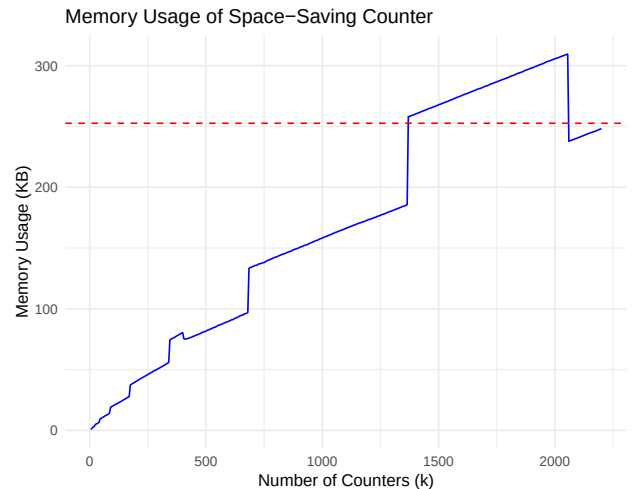


Fig. 3 - Memory usage of the Space-Saving algorithm as a function of the number of counters k . The dashed horizontal line represents the baseline memory usage of the Exact Counter.

From the data presented, we can conclude sufficient precision for detecting top-5 most frequent items can be achieved with Space-Saving Counter by using approximately only 10% of the memory used by the Exact Counter, demonstrating significant increase in efficiency of memory usage with equal results.

While the sharp increases and decreases in memory usage around $k = 1300$ and $k = 2000$ do not strictly follow the theoretical expectation of linear growth, they are more likely caused by the limitations of the `sizeof` function, which provides only an approximate measurement of memory usage. Nevertheless, the overall trend confirms that the algorithm exhibits a roughly linear increase in memory consumption as the number of counters k grows.

VI. DISCUSSION

The experimental results highlight the trade-offs inherent in approximate counting algorithms. While the Space-Saving Count demonstrated significantly reduced memory usage compared to exact counting, with accurate estimates for the most frequent items, the Probabilistic Counter did not achieve the memory savings predicted by theoretical analysis. This discrepancy is likely due to Python's memory allocation and the overhead associated with storing individual counters, even when using smaller data types. Nevertheless, when scaling the counts by the inverse of the sampling probability, the Probabilistic Counter shows that even with only 12.5% of occurrences counted, the relative

error for the most frequent towns remains below 1%. This demonstrates that probabilistic sampling is highly effective for identifying heavy hitters, while still sometimes overlooking the less frequent items in large data streams.

For the Space-Saving algorithm, the number of counters k is the critical parameter. The results show that relatively few counters—just 6.8% of the total number of unique items, are sufficient to capture all top-5 towns, demonstrating the algorithm’s accuracy. However, for very small k , the algorithm may temporarily fail to identify some of the most frequent items, highlighting the potential variability in top- k identification when resources are limited. Therefore, before deploying the algorithm in real-life applications, the choice of the parameter k should carefully balance memory constraints and the desired accuracy for top- k identification. Not selecting sufficient number of counters can result in occasional misses among the most frequent items, while increasing k quickly stabilizes the results and ensures reliable detection of the most frequent items. This emphasizes the importance of parameter tuning in practical scenarios, particularly when the dataset contains a large number of unique elements.

Memory analysis indicates that the Space-Saving Count algorithm exhibits a predictable memory usage growth pattern, while the Probabilistic Counter deviates from theoretical predictions. The Space-Saving algorithm shows an approximately linear increase in memory usage with k , consistent with theoretical expectations, although minor irregularities are observed due to measurement approximations with `sizeof`. In contrast, the Probabilistic Counter maintains a roughly fixed memory footprint per item, indicating that increasing the range of counted items does not significantly reduce memory usage. This suggests that while the Probabilistic Counter can efficiently estimate frequencies in extremely large datasets, it does not inherently provide memory savings compared to exact counting, highlighting the trade-off between sampling efficiency and absolute memory consumption. More pronounced memory savings could likely be achieved if the algorithm were implemented in a strongly typed language such as **C**, **C++**, or **Java**.

Overall, these findings suggest that approximate counting algorithms provide a practical balance between accuracy and efficiency. Both of them are well suited for detecting the **frequent** items in the large data streams, while Probabilistic Counter can struggle with rarer items. And while it did not exhibit significant memory upgrades over the exact method, the Space-Saving Count did exactly that, proving to be valid alternative when only high-frequency items are our point of interest.

VII. CONCLUSION

In conclusion, the experiments demonstrate that approximate counting algorithms offer an effective trade-off between accuracy and efficiency for analyzing large

data streams. The Space-Saving Count algorithm proved highly effective in identifying the most frequent items with a relatively small number of counters, achieving perfect top-5 detection with number of counters equal to only a fraction of number of unique items in the dataset. Its memory usage scales predictably with the number of counters, making it a practical choice when memory is severely limited.

The Probabilistic Counter, while able to estimate frequencies accurately for the most common items even when counting only 12.5% occurrences, does not inherently reduce memory usage compared to exact counting in Python, highlighting the limitations of probabilistic sampling in high-level languages. Nevertheless, it remains a viable approach for extremely large datasets where approximate results are sufficient, and relative errors for heavy hitters are minimal.

Overall, these findings suggest that algorithm selection should be guided by the specific requirements of the task: for precise identification of frequent items with memory efficiency, Space-Saving Count is preferable, whereas Probabilistic Counter provides a lightweight alternative for approximate frequency estimation in massive streams. Both algorithms highlight the importance of parameter tuning, especially the number of counters for Space-Saving, to balance memory usage and accuracy in practical deployments.

Future work should focus on testing implementations of the algorithms in programming languages that are more "low-level", since they allow higher control over memory usage. As already mentioned, good examples of languages for future work include C, C++, and Java, which offer more control over memory allocation and data types. Implementing Space-Saving Count and Probabilistic Counter in these languages could further improve memory savings (especially for Probabilistic Counter), reduce overhead associated with high-level abstractions, and allow benchmarking on larger datasets.

REFERENCES

- [1] Ahmed Metwally, Divyakant Agrawal, and A. Abbadi, "Efficient computation of frequent and top- k elements in data streams", 01 2005, pp. 398–412.
- [2] "Pympler: Python library for memory profiling", <https://pympler.readthedocs.io/>, Accessed: 29 December 2025.